AUTOMAÇÃO DE TESTES WEB

Selenium WebDriver: da sessão ao parecer

Controle um navegador em um projeto externo, observe o estado da interface e investigue uma falha antes de atribuir sua causa.

O caso Fikily

Um carrinho fictício tem subtotal de R$ 200,00. O cupom FVR10 deve mostrar Cupom aplicado, desconto de R$ 20,00 e total de R$ 180,00 quando o resultado estiver pronto.

Prática	Pergunta central
Montar	Como criar e encerrar uma sessão própria de WebDriver?
Localizar	Quais atributos identificam os controles sem depender da posição?
Esperar	O resultado está pronto ou apenas presente na página?
Investigar	A divergência vem do produto ou do instante da leitura?

Simulação não é execução real

O QA Quest não instala nem executa Selenium, código do aluno, checkout real ou serviços externos. As evidências são fixtures locais. O código ilustrativo requer um projeto Node.js configurado e uma aplicação de laboratório autorizada fora do site.

A leitura é opcional e não muda XP. Os módulos anteriores são recomendados, não pré-requisitos além da Fase 4.

01 / SESSÃO E LOCALIZADORES

Encontre o elemento e cuide da sessão.

WebDriver comunica-se com o navegador. Uma biblioteca externa faz as comparações: este exemplo usa node:assert/strict. Builder cria a sessão, By descreve como encontrar elementos e quit encerra o driver. [1, 2]

```
const { Builder, By, until } = require("selenium-webdriver");
const assert = require("node:assert/strict");

async function testCoupon() {
  let driver;
  try {
    const labUrl = process.env.FIKILY_LAB_URL;
    if (!labUrl) throw new Error("Configure o laboratório.");
    driver = await new Builder().forBrowser("chrome").build();
    await driver.get(labUrl);
  } finally {
    if (driver) await driver.quit();
  }
}
testCoupon().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

FIKILY_LAB_URL representa uma aplicação autorizada que implementa os controles do exercício. Não é uma URL pública oferecida pelo QA Quest. [2]

Controle	Localizador possível
Cupom	By.id("coupon-code")
Aplicar	By.css('[data-testid="apply-coupon"]')
Resultado	By.css('[data-testid="coupon-result"]')

Escolha pelo contrato

By.id, By.name e By.css são estratégias válidas. CSS e XPath não são automaticamente incorretos; a fragilidade vem de depender de posições ou detalhes visuais que não representam a regra. [3]

02 / INTERAÇÕES E VERIFICAÇÕES

Mensagem positiva não basta.

Localize, preencha, clique, aguarde e compare. O exemplo abaixo é código para um projeto Node.js externo, não um script executado pelo QA Quest. [2, 4]

```
const { Builder, By, until } = require("selenium-webdriver");
const assert = require("node:assert/strict");
async function testCoupon() {
  let driver;
  try {
    const url = process.env.FIKILY_LAB_URL;
    if (!url) throw new Error("Configure o laboratório.");
    driver = await new Builder().forBrowser("chrome").build();
    await driver.get(url);
    const field = await driver.findElement(By.id("coupon-code"));
    await field.sendKeys("FVR10");
    const button = await driver.findElement(
      By.css('[data-testid="apply-coupon"]')
    );
    await button.click();
    await driver.wait(
      until.elementLocated(By.css('[data-testid="coupon-result"]')),
      5000
    );
    await driver.wait(async () => {
      const result = await driver.findElement(
        By.css('[data-testid="coupon-result"]')
      );
      return (await result.getAttribute("data-state"))
        === "ready";
    }, 5000);
    const read = async (id) =>
      (await driver.findElement(By.css(`[data-testid="${id}"]`)))
        .getText();
    assert.equal(await read("coupon-message"), "Cupom aplicado");
    assert.equal(await read("coupon-discount"), "R$ 20,00");
    assert.equal(await read("coupon-total"), "R$ 180,00");
  } finally {
    if (driver) await driver.quit();
  }
}
testCoupon().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

sendKeys e click fazem a interação; getText lê o conteúdo. A comparação de mensagem, desconto e total é feita por node:assert/strict, não por uma assertion embutida no WebDriver. [1, 2]

03 / ESPERAS E EVIDÊNCIAS

Espere uma condição verificável.

until.elementLocated pode aguardar presença, mas presença não significa visibilidade nem resultado pronto. driver.wait também aceita uma função que verifique uma condição observável com timeout. Misturar esperas implícitas e explícitas pode alterar o tempo previsto. [4, 5]

Perfil	Observado	Interpretação
Referência confiável	800 ms: pronto; desconto R$ 20,00 e total R$ 180,00.	Caso aderente neste escopo.
Candidata confiável	800 ms: mensagem positiva, desconto R$ 0,00 e total R$ 200,00.	Defeito do produto: a cobrança ignora o cupom.
Referência frágil	300 ms: leitura do total antigo. 800 ms: referência correta em R$ 180,00.	Falso alarme causado por leitura prematura.

```
await new Promise((resolve) => setTimeout(resolve, 300));
const oldTotal = await driver.findElement(
  By.css('[data-testid="coupon-total"]')
).getText();
assert.equal(oldTotal, "R$ 180,00");
```

Por que o teste frágil falha?

Em 300 ms, a variável captura R$ 200,00; ela não se atualiza quando a página chega a R$ 180,00. Uma espera explícita pelo estado pronto evita atribuir esse falso alarme ao produto. [4]

Tempos e eventos são didáticos e determinísticos. Esta sequência temporal não é um log do Selenium nem uma medição do navegador.

04 / REVISÃO E PARECER

Teste confiável começa de um estado conhecido.

Prepare dados e sessão independentes entre casos. Encerre o driver em finally mesmo após uma falha posterior à criação da sessão. A documentação do Selenium recomenda evitar estado compartilhado entre testes. [2, 6]

Parecer sustentado

A referência atende ao caso preparado. A candidata confirma o cupom sem alterar o valor e deve voltar para ajuste e nova avaliação. O teste frágil precisa de sincronização por condição. Concluir a missão não aprova a suíte real nem a aplicação para produção.

Exercício de revisão

1. Por que encontrar coupon-result não prova que o desconto foi aplicado?
2. O que precisa ser conferido além da mensagem Cupom aplicado?
3. A leitura de R$ 200,00 aos 300 ms prova um defeito na referência?
4. Por que driver.quit deve estar em finally?
5. Por que não misturar automaticamente esperas implícitas e explícitas?

Critérios de correção

1. Presença não prova estado pronto nem regra de negócio.
2. Conferir desconto R$ 20,00 e total R$ 180,00.
3. Não: a referência fica correta aos 800 ms; o teste leu cedo demais.
4. Para encerrar uma sessão criada mesmo se houver falha.
5. A combinação pode produzir tempos de espera imprevisíveis.

O QA Quest usa escolhas e fixtures locais. Em um projeto real, ambiente, dados, runner, navegadores e critérios de aprovação precisam de configuração e avaliação próprias.

05 / REFERÊNCIAS E PRÓXIMOS PASSOS

Continue com fontes gratuitas.

Conteúdo autoral didático, com exemplos próprios e paráfrases. Consulte a documentação correspondente à versão usada no seu projeto. Links verificados em 09/11/2026.

[1] **Componentes e papel do WebDriver**
https://www.selenium.dev/documentation/overview/components/

[2] **Primeiro script em JavaScript**
https://www.selenium.dev/documentation/webdriver/getting_started/

[3] **Estratégias de localização**
https://www.selenium.dev/documentation/webdriver/elements/locators/

[4] **Estratégias de espera**
https://www.selenium.dev/documentation/webdriver/waits/

[5] **API JavaScript de WebDriver**
https://www.selenium.dev/selenium/docs/api/javascript/WebDriver.html

[6] **Evitar estado compartilhado**
https://www.selenium.dev/documentation/test_practices/encouraged/avoid_sharing_state/

Próxima prática responsável

Em um projeto de testes autorizado, configure o Selenium WebDriver para JavaScript, uma aplicação de laboratório e um runner adequado. Comece por um caso pequeno e independente. Não use contas, dados ou sistemas de terceiros sem permissão.

Versão 1.0 · 09/11/2026. O laboratório do QA Quest é uma simulação local e não executa Selenium ou código do aluno.